Personal AI Web Research Agent with Ollama and Qwen/Llama3

Table of Contents

Background

Most of us have used ChatGPT or Claude to send queries to a large language model. You've probably also seen hallucinations in the response when the model didn't know something, sometimes because its knowledge was out of date.

With the rise of tool calling, LLMs can now use tools to search the web for the latest information. They can then bring that information into context and use it to generate an output, summarize results, and extract key points from retrieved sources.

In this tutorial, I'll show you how I built a personal research agent that searches the internet for any topic and uses local LLM to summarize what it finds. It runs entirely on my own machine to preserve privacy and has no API costs. So, it's completely free.

You'll need Ollama installed on your machine and a free Ollama account. The tutorial works on macOS, Windows, and Linux. I'm using a MacBook Pro with 32 GB of RAM, but you can run this on a lower-memory machine by choosing a smaller Qwen model from Ollama.

Motivation and Architecture

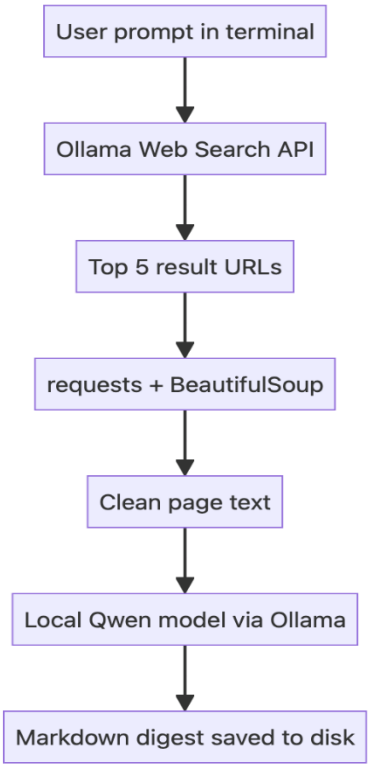
The motivation behind this project is to have agents running on my machine that can handle a variety of tasks every day. I can spin off agents to create a daily digest of AI news, surface the latest world events, or look for new job postings.

Running a local LLM also means none of these queries leave my machine. My research history stays private, and there are no per-query API costs to worry about.

For this project, we'll use Ollama web search for retrieval and local Qwen LLM for summarization (rather than rely on hosted chat tools like ChatGPT or Claude). The system diagram below shows how the agent works.

When run in the terminal, the agent asks the user what they want to research. It then calls the Ollama web search API to fetch the top 5 results for the query, downloads each of those pages, and extracts the readable text.

The extracted content from all five pages is sent to the local Qwen model along with the user's prompt and a system prompt: "*Use these web results and page contents to answer in Markdown format*." The model's response is then saved as a Markdown file on disk.



Step 1: Install Ollama and Get an API Key

To get started, install the Ollama application and create an account to get an API key. The free tier of Ollama will suffice for this tutorial.

Once you have the key, place it in an environment variable:

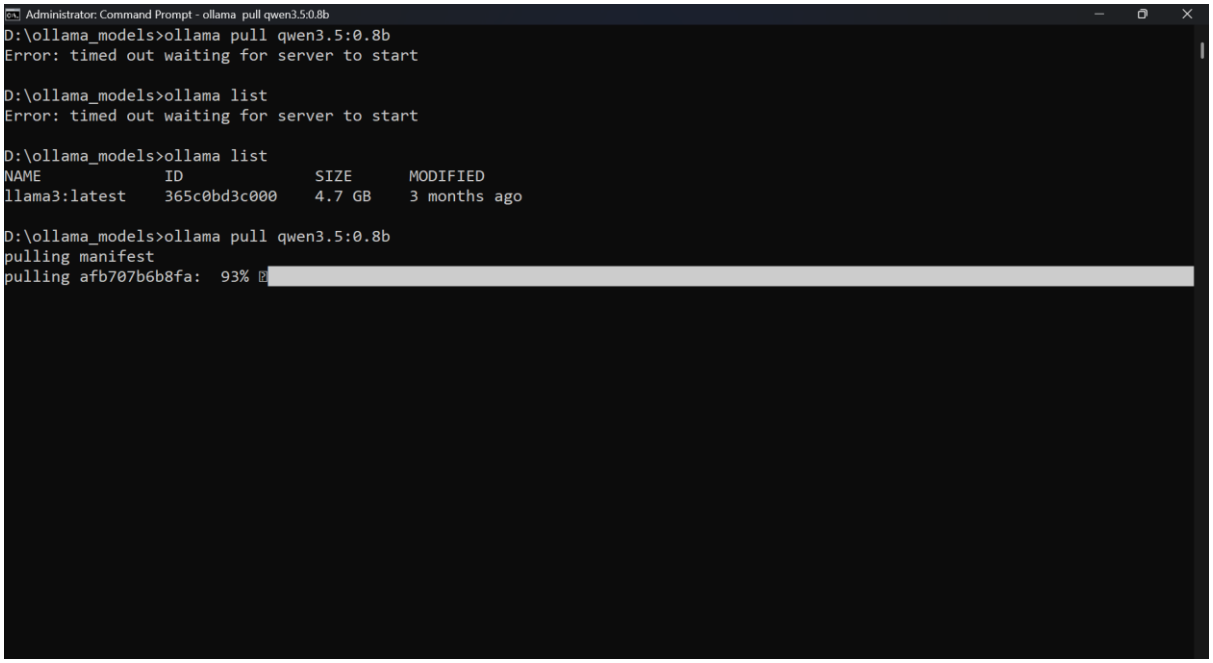
Step 2: Pull the Qwen Model

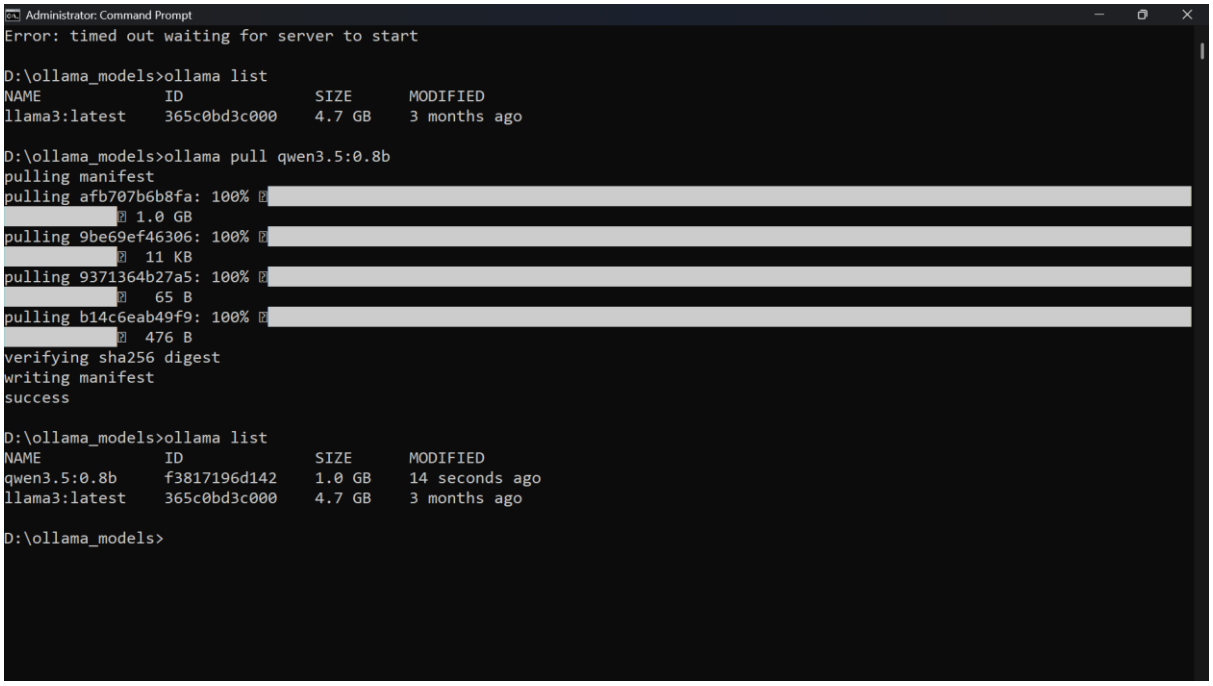
We'll use Qwen for this tutorial, an open-weight model that's currently one of the best smaller sized models available.

I'm using the 4-billion-parameter variant because it follows structured prompts well and runs on a laptop without a dedicated GPU. There are other sizes like 2b or 9b available.

To use Qwen3.5:4b locally, install it using Ollama. The 4b model size is around 3.4 GB on my machine. If your machine has lower RAM, you can use qwen3.5:0.8b instead of the 4b model.

ollama pull qwen3.5:4b



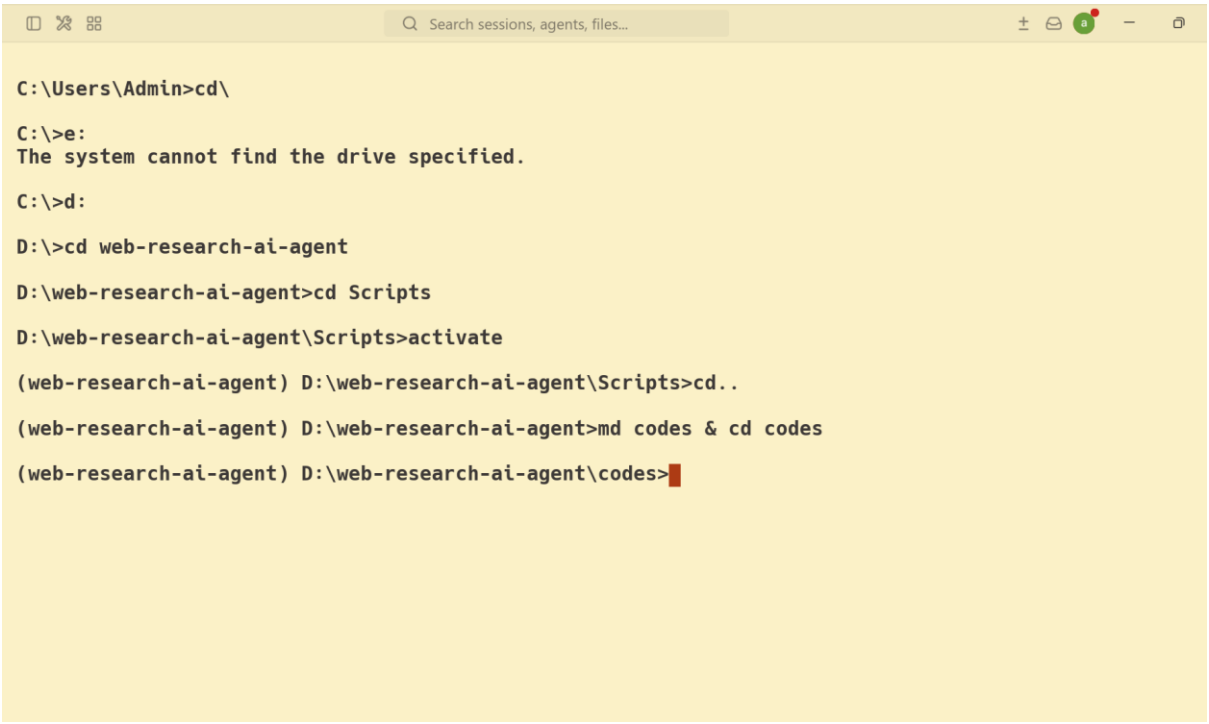


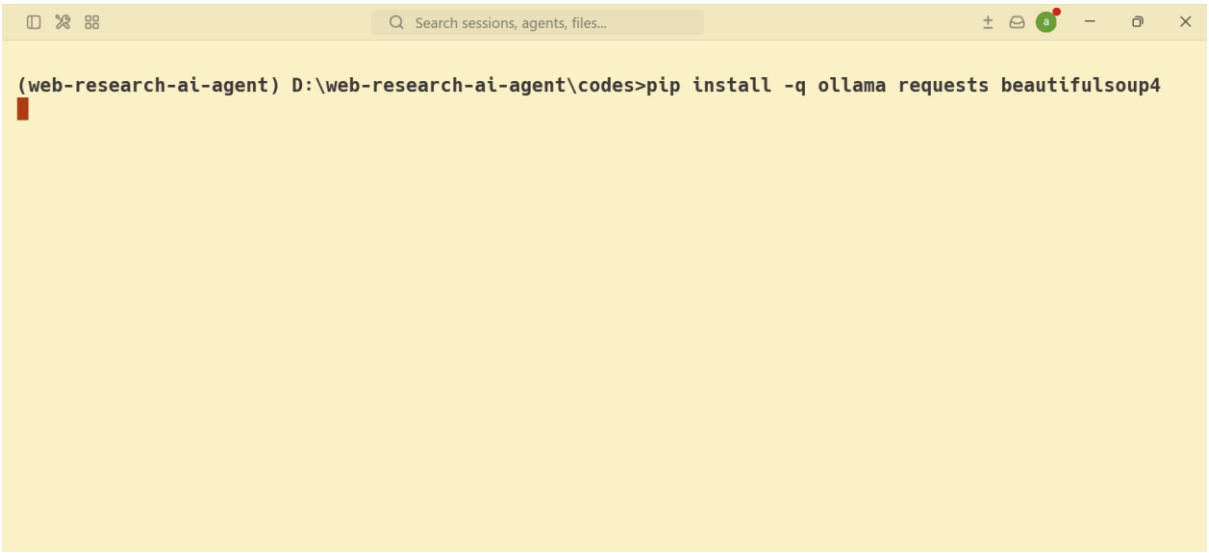
Step 3: Install Python Dependencies

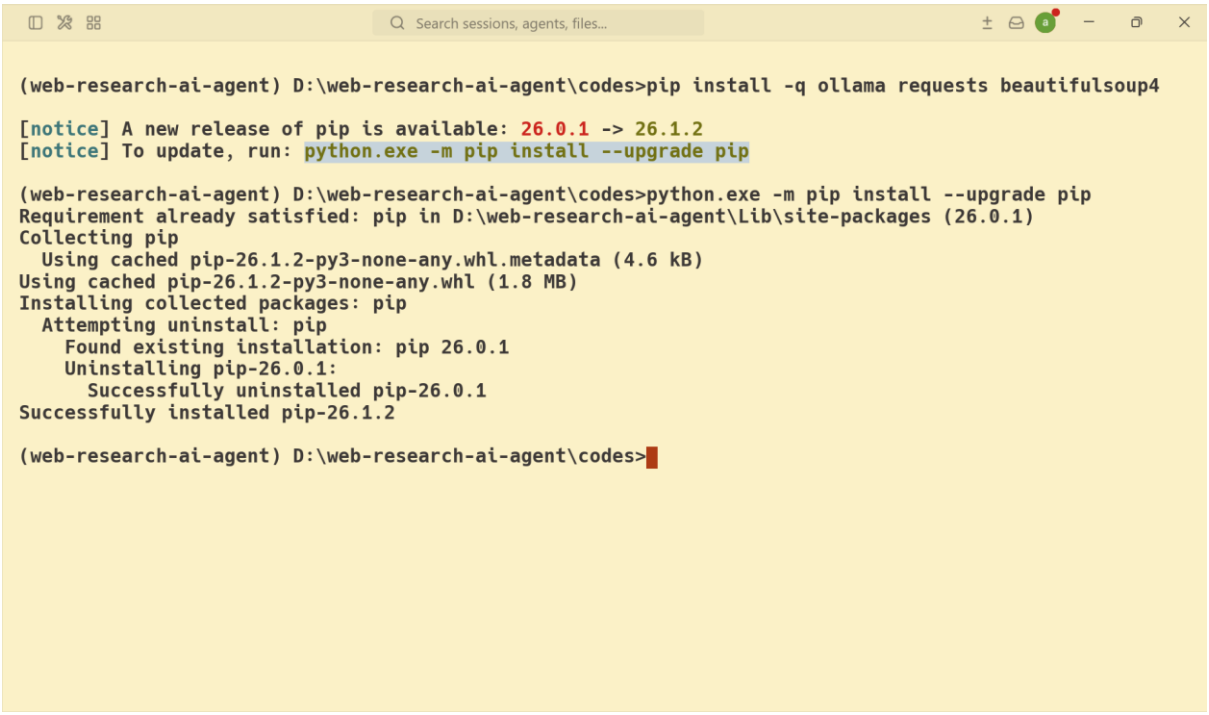
python3 -m venv venv

source venv/bin/activate

pip install ollama requests beautifulsoup4


Search sessions, agents, files...
C:\Users\Admin>cd\
C:\>e:
The system cannot find the drive specified.
C:\>d:
D:\>cd web-research-ai-agent
D:\web-research-ai-agent>cd Scripts
D:\web-research-ai-agent\Scripts>activate
(web-research-ai-agent) D:\web-research-ai-agent\Scripts>cd..
(web-research-ai-agent) D:\web-research-ai-agent>md codes & cd codes
(web-research-ai-agent) D:\web-research-ai-agent\codes>


Search sessions, agents, files...
(web-research-ai-agent) D:\web-research-ai-agent\codes>pip install -q ollama requests beautifulsoup4



Step 4: Write the Agent Code

The below Python code does four things: it takes a research prompt from the terminal, calls Ollama's web search API for the top 5 results, downloads the webpages using Requests and cleans each page's text using BeautifulSoup, then sends everything to a local Qwen model with an instruction to summarize in Markdown. Finally, it saves the result to a timestamped .md file.

Save the code in your research_agent.py file.

```
import os

import json

import requests

import ollama

from bs4 import BeautifulSoup

from datetime import datetime

from pathlib import Path


API_KEY = os.getenv("OLLAMA_API_KEY")
```

```
SEARCH_URL = "https://ollama.com/api/web_search"
MODEL = "qwen3.5:4b"
#MODEL = "llama3:latest"

# Search web using Ollama web search
def search_web(query):
    response = requests.post(
        SEARCH_URL,
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={"query": query, "max_results": 5},
        timeout=30,
    )
    response.raise_for_status()
    return response.json().get("results", [])

# Fetch full web page content
def fetch_text(url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
    except requests.RequestException as e:
        return ""
    soup = BeautifulSoup(response.text, "html.parser")
    for tag in soup(["script", "style", "nav", "footer"]):
        tag.decompose()
    return soup.get_text(separator="\n", strip=True)
```

```
def main():
    user_prompt = input("Enter your prompt: ").strip()
    if not user_prompt:
        print("Prompt cannot be empty.")
        return

    results = search_web(user_prompt)

    # For each url in web search result, fetch full content
    pages = []
    for item in results:
        url = item.get("url")
        if not url:
            continue

        print(f"Fetching: {url}")
        page_text = fetch_text(url)

        pages.append({
            "title": item.get("title", ""),
            "url": url,
            "snippet": item.get("content", ""),
            "page_text": page_text,
        })

    # Prompt to send to Qwen model with web data
    prompt = f"""
    User request:
```

```
    {user_prompt}

    Use these web results and page contents to answer in markdown format.

    Data:
    {json.dumps(pages, ensure_ascii=False)}
    """

    # Invoke local Qwen model
    response = ollama.chat(
        model=MODEL,
        messages=[{"role": "user", "content": prompt}],
    )

    digest = response.message.content

    # Build a unique filename using today's date and time
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"digest-{timestamp}.md"

    # Save the digest to disk
    with open(filename, "w") as f:
        f.write(digest)

    print(f"Saved to digest")

if __name__ == "__main__":
```

```
main()
```

Step 5: Run the Agent

python research_agent.py

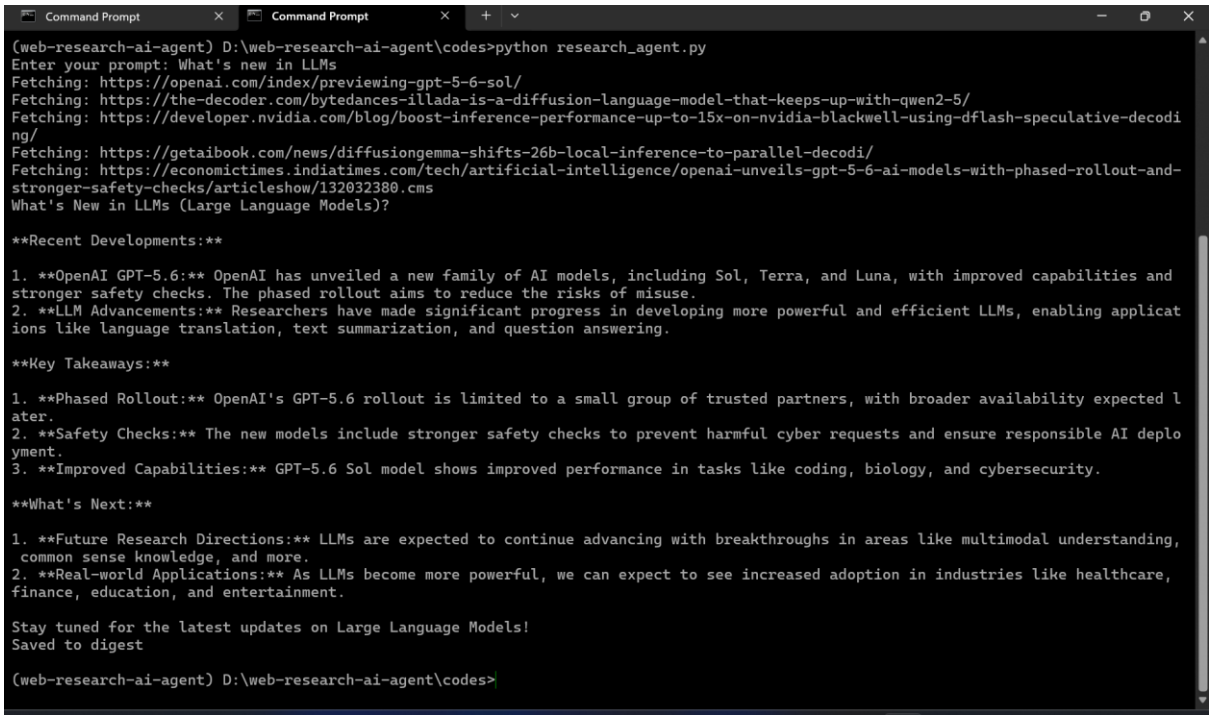
The script will prompt you to enter the topic you'd like to research.

Sample Output

The summarized digest is saved as a timestamped Markdown file. The agent also prints the source URLs as it fetches them.

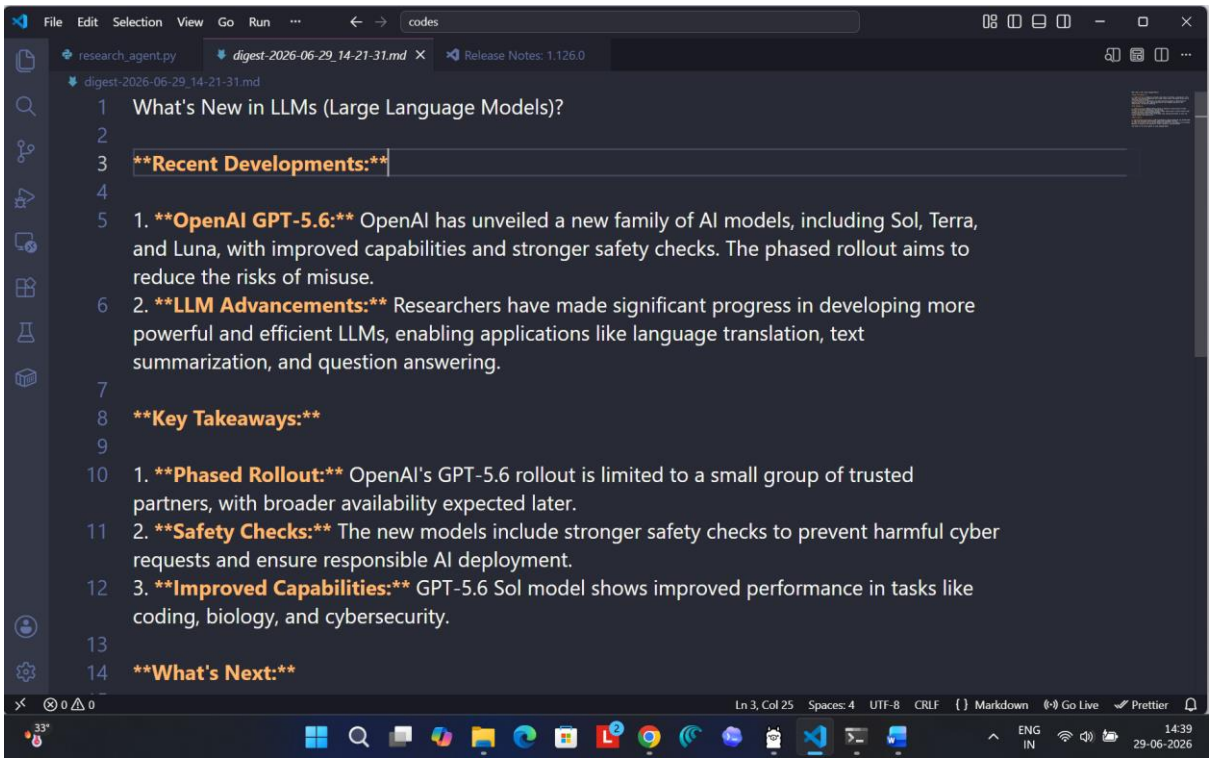
Before trusting the summary, skim it and spot-check a claim or two against the original source. Local models are smaller than hosted frontier models and tend to hallucinate more. So spot-checking can help with accuracy.

As a test run, I asked the research agent: "**What's new in LLMs**" and it fetched 5 web pages as seen below:



The digest came out reasonably well-structured for a 4B local model. It's organized into sections with all the relevant data from the sources. I spot-checked the summary and it was accurate.

Here's what it produced:



Conclusion

In this tutorial, you learned how to build a personal AI web research agent that searches the web, summarizes results with a local LLM, and saves a Markdown digest. All this runs on your own machine with no data leaving your laptop. You have full control over the model and prompts without any API costs.

From here, you can try new prompts to research different topics, tweak the system prompt to change the output, swap in other local models like Qwen 3.6 or Mistral, or extend the script to fit your own workflow.